A Lean-Checked Potential Calculus for Bounded-Cost LLM-Agent Workflows: A Sequential Core

Hernán Inverso

June 12, 2026

Preprint, v3 — 12 June 2026

Relation to the closest work (Khan [10]). Khan provides an empirical catalogue of 63 LLM-agent budget-overrun incidents and an affine (Rust borrow-checker) mitigation; its aggregate bound (Proposition 1) is pen-and-paper and conditional on an estimator assumption, and binary-level cap-soundness remains open (Conjecture 1). We instead **mechanize (Lean 4) an operational aggregate cost bound** for a sequential potential calculus under an explicit per-call cap axiom, with a per-provider billing analysis. We do *not* address binary-level soundness, and our affine no-double-spend layer is deliberately thinner than Rust-style ownership.

Abstract

Autonomous LLM-agent workflows consume tokens, tool calls and money in loops, and current safeguards are *runtime* and *fallible*: a per-call cap exists, but the *project-level* budget only notifies (OpenAI's project budget now warns rather than halts; Azure recommends "implement your own logic"), and nothing bounds the **aggregate** across many capped calls — a single in-budget call can still push a multi-step run over its total after it completes. We give a small **affine calculus with numeric potential**, in the style of Hofmann–Jost Automatic Amortized Resource Analysis (AARA), with seven constructs mirroring real frameworks and a delegation rule whose linear potential split is adapted from resource-aware session types. This is a **deliberately small sequential core**: integer costs, no concurrency/retry/expected-cost. Our **machine-checked (Lean 4) cost-soundness theorem** states that a well-typed workflow's accumulated gas never exceeds its declared potential at *every reachable configuration* — a termination-free safety invariant (it covers every finite prefix) **under the cap axiom** (per-call cost $\leq$ the declared cap), which §3.2 shows holds for some providers and degrades for others. The theorem is mechanized both for the scalar core *and*, in a companion file, for an **arbitrary-dimension resource vector** $\langle \text{tokens}, \text{calls}, \$ \rangle$ (`vsteps_sound`, all k), so the multi-resource reading is machine-checked, not pen-and-paper. For the presented fragment we additionally prove progress and strong normalization: within the core semantics a well-typed workflow *terminates within* $\leq P$ tokens (under the cap axiom and a declared-window assumption on re-sent context) — a liveness-flavored property no kill-switch provides. A separate Lean file mechanizes a **minimal affine no-double-spend ledger invariant**; full borrow-style ownership remains future work. We then contribute a **per-provider billing analysis** (the most practically useful part): the operational axiom that a per-call cap is a hard *billing* ceiling on output (including hidden reasoning tokens) is **parameter-specific, not provider-specific**. It holds for OpenAI Responses (`max_output_tokens`), Azure/OpenAI reasoning (`max_completion_tokens` — reasoning tokens are inside `completion_tokens`), and Anthropic standard (`budget_tokens` < `max_tokens`), but **degrades** for Anthropic interleaved/adaptive thinking (budget may exceed `max_tokens`) and Gemini when `maxOutputTokens` is set without pinning the separate `thinkingBudget` (§3.2, primary docs). Finally, the calculus is realized as **`gasket`, an open-source static checker**, and validated by an **empirical study of 254 real agent workflows** from 45 public repositories (§6): 76% map onto the calculus and 88% of cyclic workflows rely on a *framework default* (often a

coarse 1000-superstep LangGraph limit, too loose to be a meaningful budget) for their only budget bound. To our knowledge this is the **first Lean-mechanized aggregate cost-soundness theorem specialized to per-call-capped LLM-agent workflows**; the proof technique is textbook AARA, the contribution is the instantiation, the provider-billing semantics, and the tool + study.

1 Introduction

The pain. LLM-agent frameworks reduce a workflow to a small graph of model calls, tool calls, branches, bounded loops and sub-agent delegations; each step costs tokens and money. Cost is dominated by *re-sent context* (a stateless API re-sends the system prompt, tool definitions and the whole conversation each step), which grows superlinearly. The problem is documented with primary sources: Cursor issued public refunds for surprise agent bills (July 2025); **OpenAI eliminated its hard budget cap** — a project budget now only notifies, "API requests will continue to be processed without interruption" [OpenAI Help Center 9186755, accessed Jun 2026]; **Azure OpenAI provides no hard spending cap** and officially recommends "implementing your own logic in your application to halt requests" [Microsoft Q&A, accessed Jun 2026]. The `TALE` study [7] documents *token elasticity*: under a tight budget the model emits *more* tokens, so prompting is not a ceiling.

State of the art: runtime and fallible. Observability vendors (Langfuse, Helicone, Portkey) sell tracking and alerts, not enforcement; LiteLLM gives runtime per-key budgets for free. *Agent Contracts* [1] specifies multi-dimensional resource bounds with conservation laws but enforces them *at runtime* and admits a single call's cost is known only on completion. Khan [10] is the first to bring *static* typing to bear: an affine (Rust borrow-checker) mitigation whose source-level ownership integrity is enforced by the borrow checker (pen-and-paper, not mechanized), with the aggregate cost bound given as Proposition 1 (conditional on estimator assumption A1) and validated empirically, and binary-level soundness left open (Conjecture 1).

This paper. We provide an *operational* cost-soundness result via a potential-based calculus — complementary to Khan's affine ownership discipline, and orthogonal to his open binary-level gap.

Contributions.

1. **A calculus** (§2): an affine workflow calculus with numeric potential over the semiring $\mathbb{N}^k$ (presented for $k = 1$ for readability, and *mechanized both for $k = 1$ and for arbitrary k* — see contribution 2), with seven constructs mirroring real frameworks, including a delegation rule whose linear potential split is adapted from resource-aware session types.

2. **A machine-checked (Lean 4) cost-soundness theorem** (§4): well-typed $\Longrightarrow$ gas $\leq$ declared potential at every *reachable configuration* (hence every finite prefix of any trace), *under the cap axiom* (§3), proved as a termination-free safety invariant. *Scope of the mechanization, stated plainly:* the Lean checks (i) `steps_sound` (the §4 bound), (ii) `hastype_iff_pot` (Lemma 0 — the §3 relational rules equal pot+`WellTyped`), (iii) `step_decreases` (every step strictly decreases the §5 (`Wm`, `Sz`) measure), and (iv) `vsteps_sound` — the *same* cost-soundness theorem for an **arbitrary-dimension resource vector** $\mathrm{Res}\, k = \mathrm{Fin}\, k \to \mathbb{N}$ with pointwise $\leq$/+/max (companion file `lean/TypedResourcesVec.lean`), so the multi-resource $\langle \text{tokens}, \text{calls}, \$ \rangle$ reading is machine-checked rather than a pen-and-paper lift; a companion file `lean/Affine.lean` checks (v) `no_double_spend` (the §8 affine layer). `steps_sound`, `hastype_iff_pot`, and `vsteps_sound` depend only on the kernel axioms `[propext, Quot.sound]` (`step_decreases` additionally on `Classical.choice`); the artifact ships the `#print axioms` output for each. It does **not** mechanize the final well-foundedness

step of strong normalization, progress, the fusion of the two layers into one syntax, or the §3.2 billing facts (these remain on paper). The credit-invariant technique itself is textbook AARA [9], and AARA has been mechanized before (e.g. Carbonneaux–Hoffmann–Shao, certified resource bounds [2]); **our novelty is not the proof technique but (i) its first machine-checked instantiation to the agent-workflow/per-call-cap setting and (ii) the billing analysis below**. Khan establishes the aggregate bound via Proposition 1 + empirics, unmechanized.

3. **A static-vs-runtime observation** (§5): the type rejects unbounded loops *ahead of time*, gives static compositional conservation for sequential/nested delegation, and — via strong normalization — certifies completion within budget. (No-double-spend is *not* part of this potential fragment; it is the separate affine handle layer of §8, also mechanized — `no_double_spend`.)

4. **A per-provider billing analysis** (§3.2): the operational axiom is *parameter-specific*. It holds for OpenAI Responses, Azure/OpenAI reasoning (`max_completion_tokens` bounds reasoning+output), and Anthropic standard; it degrades for Anthropic interleaved/adaptive thinking and for Gemini when `maxOutputTokens` is set without pinning `thinkingBudget`. We prescribe the correct cap parameter.

Scope is stated throughout: this is the minimal fragment. The affine no-double-spend layer is mechanized separately (§8) and the multi-resource vector metatheory is mechanized for arbitrary k (§4); concurrency (`par`/`retry`), the fusion of the two layers, and expected-cost are roadmap (§8), not claimed.

1.1 Delta vs the closest prior work

	Khan [10]	Resource-Bounded Type Theory [14]	**This paper**
mechanism	affine ownership (Rust borrow checker)	graded modalities, abstract resource lattice	**AARA numeric potential**
proofs	pen-and-paper (no proof assistant)	syntactic, recursion-free	**machine-checked (Lean 4)**
cost guarantee	Proposition 1 (cond. on A1) + empirical, not proved	syntactic cost-soundness, recursion-free	**operational cost-soundness, termination-free safety invariant**
resources	one budget	abstract lattice (time/mem/gas)	**⟨tokens, calls, $⟩ tuple**
delegation	—	—	**compositional split (session-type style)**
LLM coupling	dollar cap, no reasoning models	none (general resources)	**cap axiom + per-provider billing + framework map**
left open	binary-level soundness (Conjecture 1)	recursion	concurrency, full ownership (Γ/move/aliasing), layer fusion, expected-cost

Our novelty is the *coupling* of amortized potential to the agent setting — the cap axiom, the $\langle \text{tokens}, \text{calls}, \$ \rangle$ tuple, compositional delegation, and the per-provider billing analysis — not the potential method itself, which we reuse from AARA.

2 The calculus (presented fragment: one resource, integer costs, pure potential)

We present the fragment used in the metatheory: a single resource, integer costs, *pure numeric potential* $p \in \mathbb{N}$ (no affine context Γ). The vector form replaces $\mathbb{N}$ by $\mathbb{N}^k$ and $\geq/-$ pointwise; all rules are unchanged. The affine *handle* layer that yields no-double-spend is a separate calculus, mechanized in its core form in §8; we keep this base case minimal so the theorem is checkable.

Syntax.

e	::=	$\mathsf{call}(c) \mid \mathsf{tool}(c)$	model/tool call, declared cap c (the API token ceiling)
	$\mid$	skip	terminated workflow (value)
	$\mid$	$e_1 \,;\, e_2$	sequence
	$\mid$	$\mathsf{if}\ e_1\ e_2$	branch (nondeterministic choice of a branch)
	$\mid$	$\mathsf{loop}(n, e)$	bounded loop, fuel n a literal; no fuel $\Rightarrow$ no rule $\Rightarrow$ does not type
	$\mid$	$\mathsf{delegate}(q, e)$	sub-agent with transferred potential q (linear split of parent)

We write e *typable* iff $\exists p, p'.\ p \vdash e : \diamond ;\ p'$.

Judgment. $p \vdash e : \diamond ;\ p'$: with incoming potential p and residual p'.

Typing rules.

$$\text{(T-Call)}\ \frac{p \geq c}{p \vdash \mathsf{call}(c) : \diamond ;\ p - c} \qquad \text{(T-Skip)}\ \frac{}{p \vdash \mathsf{skip} : \diamond ;\ p}$$

(T-Tool) is identical to (T-Call).

$$\text{(T-Seq)}\ \frac{p \vdash e_1 : \diamond ;\ p_1 \qquad p_1 \vdash e_2 : \diamond ;\ p_2}{p \vdash e_1 ; e_2 : \diamond ;\ p_2} \qquad \text{(T-If)}\ \frac{p \vdash e_1 : \diamond ;\ p_1 \qquad p \vdash e_2 : \diamond ;\ p_2}{p \vdash \mathsf{if}\ e_1\ e_2 : \diamond ;\ \min(p_1, p_2)}$$

In (T-If) the *same* input p goes to both branches.

$$\text{(T-Loop)}\ \frac{p_b \vdash e : \diamond ;\ p'_b \qquad b := p_b - p'_b \qquad p \geq n \cdot b}{p \vdash \mathsf{loop}(n, e) : \diamond ;\ p - n \cdot b}$$

Fuel n is a literal; no fuel $\Rightarrow$ no rule.

$$\text{(T-Del)}\ \frac{q \vdash e : \diamond ;\ q' \qquad p \geq q}{p \vdash \mathsf{delegate}(q, e) : \diamond ;\ p - q}$$

Linear split: the parent transfers q; conservative.

Every rule is *exact*: the residual is a deterministic function of the inputs, not a slack inequality. ((T-Call) residual $p - c$, (T-Skip) p, (T-Seq) p_2, (T-If) $\min(p_1, p_2)$, (T-Loop) $p - n \cdot b$, (T-Del) $p - q$.) Exactness is what makes Lemma 0 hold; an earlier draft wrote (T-Loop)/(T-Del) with a free residual p' in the premise ($p \geq n \cdot b + p'$), which would have made $p - p'$ ambiguous (e.g. $\mathsf{loop}(0, e)$ at $p = 10$ could derive any residual $0 \ldots 10$) — that version is **wrong** and is corrected here. In (T-If) both branches are typed from the **same** input p; the residual is $\min(p_1, p_2)$ (worst residual) and the cost is $b_{\text{if}} = \max(b_{e_1}, b_{e_2})$ (worst branch). The cost b in (T-Loop) is well-defined because, with exact rules, every typable e has a unique cost $b_e = \text{pot}(e)$ independent of the incoming potential (Lemma 0, now immediate by induction).

Worked example. A research agent: call the model, loop three times over (tool + model), then branch into either a delegated sub-agent or a cheap finish:

$$\begin{aligned} \mathit{research} = {} & \mathsf{call}(4000)\ ; \\ & \mathsf{loop}(3,\ \mathsf{tool}(1000)\ ;\ \mathsf{call}(2000))\ ; \\ & \mathsf{if}\ (\mathsf{delegate}(3000,\ \mathsf{call}(2500)))\ (\mathsf{call}(1200)) \end{aligned}$$

$\mathrm{pot}(\mathit{research}) = 4000 + 3 \cdot (1000 + 2000) + \max(3000, 1200) = 4000 + 9000 + 3000 = 16000$. By §4.4, **every execution of *research* consumes ≤ 16000 units** (tokens, under the cap axiom). A typing budget of $p = 16000$ types it with residual 0; $p = 15999$ is rejected at check time. Replace $\mathsf{loop}(3, \cdot)$ with a fuel-free $\mathsf{loop}(\cdot)$ ("retry until done") and it does not type at all — the runaway is a static error, not a runtime overspend. This is exactly what the `gasket` checker (§6) computes over real LangGraph/CrewAI graphs.

3 Instrumented semantics and the billing axiom

3.1 Small-step semantics with monotone gas

Configurations $\langle e, g \rangle$, gas $g \in \mathbb{N}$. The **operational axiom** is the only place provider behavior enters: $\langle \mathsf{call}(c), g \rangle \to \langle \mathsf{skip}, g + a \rangle$ and $\langle \mathsf{tool}(c), g \rangle \to \langle \mathsf{skip}, g + a \rangle$ for some $0 \leq a \leq c$ — the API enforces actual cost $\leq$ the declared cap (it truncates and bills at most c), *not* the model's obedience (token elasticity shows the model does not obey budgets in the prompt). Remaining rules:

$$\mathsf{skip}\,;\, e \to e \qquad e_1; e_2 \to e_1'; e_2 \ \ (\text{if } e_1 \to e_1') \qquad \mathsf{if}\ e_1\ e_2 \to e_i$$

$$\mathsf{loop}(n+1, e) \to e\,;\, \mathsf{loop}(n, e) \qquad \mathsf{loop}(0, e) \to \mathsf{skip} \qquad \mathsf{delegate}(q, e) \to e$$

Input cost (re-sent context, ∼62% of the bill in practice) is folded into $c = W + \mathit{cap}_{\mathit{out}}$, with a **declared window** annotation W.

> **Modeling assumption (not a derived lemma).** Setting $c = W + \mathit{cap}_{\mathit{out}}$ with fuel-bounded loops is sound *iff* W is a worst-case bound on the re-sent context at *any* iteration (a full window). We treat W as a declared annotation, not inferred; under it, accumulated context never exceeds W per call, so per-call cost $\leq c$. Loops with strictly growing context (the source of several incidents in Khan's catalogue) require $W =$ the maximal-window cost, which is conservative (over-reservation). Inferring tighter W is future work.

The gas g is a global additive counter the semantics never reads — hence invariance under shifting the initial gas (§4.4).

3.2 When the axiom holds: per-provider billing (sample, primary docs accessed June 2026)

The cap axiom requires the declared per-call parameter to be a hard *billing* ceiling on **billed output tokens, including hidden reasoning/thinking tokens** (which every provider bills as output). Whether a given parameter achieves this is **parameter-specific, not provider-specific** — the same vendor both satisfies and breaks the axiom depending on which knob you set. Getting this right is a prescriptive contribution prior work lacks. This is a *sample*, not a complete characterization, and these APIs are volatile (each row cites the primary doc consulted):

Provider / mode	Cap parameter	Bounds billed reasoning+output?	Cap axiom
OpenAI Responses	`max_output_tokens`	**yes** — `reasoning_tokens` ⊆ `output_tokens`, bounded	**holds**
Azure/OpenAI Chat, reasoning (o-series/GPT-5)	`max_completion_tokens`	**yes** — `reasoning_tokens` ∈ `completion_tokens_details`, capped[1]	**holds**
Anthropic standard	`max_tokens` (with `budget_tokens` < `max_tokens`)	**yes** — `max_tokens` caps total output (thinking+text)[2]	**holds**
Anthropic interleaved / adaptive thinking	`max_tokens`	**no** — "`budget_tokens` can exceed `max_tokens`... across all thinking blocks"[2]	**degrades**
Google Gemini 2.5/3, thinking on	`maxOutputTokens` *alone*	**no** — thinking billed as output but governed by a *separate* `thinkingBudget`/`thinkingLevel`[3]	**degrades** unless `thinkingBudget` is also pinned
any reasoning model	legacy `max_tokens`	n/a — ignored/unsupported on o-series; must use the param above	**n/a (misconfig)**

[1] Azure Foundry "reasoning models" doc: reasoning tokens are part of `completion_tokens` and `max_completion_tokens` is the cap (Chat); `max_output_tokens` on Responses. [2] Anthropic extended-thinking doc: standard requires `budget_tokens` < `max_tokens` and `max_tokens` limits total output; interleaved explicitly lifts this. [3] Gemini thinking doc: "response pricing is the sum of output tokens and thinking tokens"; thinking is steered by `thinkingBudget` (2.5) / `thinkingLevel` (3), distinct from `maxOutputTokens`.

The earlier intuition that *reasoning models* uniformly break the cap is wrong — OpenAI/Azure reasoning *do* bound billed reasoning via the completion cap; what actually breaks it is (a) Anthropic interleaved/adaptive thinking, (b) Gemini with `maxOutputTokens` set but `thinkingBudget` unpinned, and (c) using a legacy/wrong parameter. **Corollary:** certify in **tokens/tool-calls** (stable) and map to dollars as a separate layer; the dollar bound is a guarantee exactly where the correct per-call parameter is pinned, and degrades on the three failure modes above.

4 Cost-soundness

We prove safety (the bound holds on every trace) and, for this fragment, progress + strong normalization (§5).

Two equivalent presentations. The on-paper development below uses the **relational** judgment $p \vdash e : \diamond ; p'$ (potential p in, residual p' out). The Lean mechanization uses the equivalent **functional** presentation: a closed-form $\mathrm{pot} : \mathrm{Expr} \to \mathbb{N}$ (the minimal potential the rules consume) plus a `WellTyped` side-condition for `delegate`; a budget b suffices iff $\mathrm{pot}\, e \le b$, with residual $b - \mathrm{pot}\, e$. The two agree rule-by-rule — pot is exactly what the residual rules compute — so a proof in either transfers; we mechanize the functional one because it makes the induction a direct credit ($gas + \mathrm{pot}$) invariant. Lemmas 0–2 below are the relational-side justification of that equivalence.

4.1 Lemma 1 (weakening — raise the input)

Lemma. *If $p \vdash e : \diamond ; p'$ then $p + r \vdash e : \diamond ; p' + r$ for all $r \ge 0$.*

Proof. Induction on the derivation; each rule passes the extra r from incoming to residual. ■

4.2 Lemma 0 (exact characterization — relational $\iff$ functional)

Define the closed-form potential $\mathrm{pot} : \mathrm{Expr} \to \mathbb{N}$ by $\mathrm{pot}(\mathsf{skip}) = 0$, $\mathrm{pot}(\mathsf{call}\ c) = \mathrm{pot}(\mathsf{tool}\ c) = c$, $\mathrm{pot}(e_1; e_2) = \mathrm{pot}(e_1) + \mathrm{pot}(e_2)$, $\mathrm{pot}(\mathsf{if}\ e_1\ e_2) = \max(\mathrm{pot}\, e_1, \mathrm{pot}\, e_2)$, $\mathrm{pot}(\mathsf{loop}(n, e)) = n \cdot \mathrm{pot}(e)$, $\mathrm{pot}(\mathsf{delegate}(q, e)) = q$. With the exact rules, the relational judgment is *completely determined* by it:

Lemma. *$p \vdash e : \diamond ; p'$ iff $p \geq \mathrm{pot}(e)$ and $p' = p - \mathrm{pot}(e)$ (for well-typed e; well-typedness is the delegate side-condition $\mathrm{pot}(\mathit{child}) \leq q$).*

Proof. By structural induction on e. $\mathsf{call}(c)/\mathsf{tool}(c)$: derivable iff $p \geq c = \mathrm{pot}$, residual $p - c$. skip: always, $\mathrm{pot} = 0$, residual p. $e_1; e_2$: by IH $p_1 = p - \mathrm{pot}(e_1)$ and (composing) $p_2 = p_1 - \mathrm{pot}(e_2) = p - (\mathrm{pot}\, e_1 + \mathrm{pot}\, e_2) = p - \mathrm{pot}(e_1; e_2)$, needing $p \geq \mathrm{pot}\, e_1$ and $p_1 \geq \mathrm{pot}\, e_2$, i.e. $p \geq \mathrm{pot}(e_1; e_2)$. $\mathsf{if}\ e_1\ e_2$: same input p, residual $\min(p - \mathrm{pot}\, e_1, p - \mathrm{pot}\, e_2) = p - \max(\dots) = p - \mathrm{pot}(\mathsf{if})$, needing $p \geq \max = \mathrm{pot}(\mathsf{if})$. $\mathsf{loop}(n, e)$: $b = \mathrm{pot}(e)$ by IH (unique!), residual $p - n \cdot \mathrm{pot}(e) = p - \mathrm{pot}(\mathsf{loop})$, needing $p \geq n \cdot \mathrm{pot}(e)$. $\mathsf{delegate}(q, e)$: needs $q \geq \mathrm{pot}(e)$ (child types) and $p \geq q = \mathrm{pot}(\mathsf{delegate})$, residual $p - q$. ∎

Corollary (Cost is fungible)**.** *$b_e := p - p' = \mathrm{pot}(e)$, independent of the incoming p.*

This lemma is now ***machine-checked****: `lean/TypedResources.lean` defines the relational `HasType` and proves `hastype_iff_pot` : `HasType` $p\ e\ p' \leftrightarrow$ (`WellTyped` $e \wedge$ `pot` $e \leq p \wedge p' = p -$ `pot` e) (the `WellTyped` conjunct is essential — it carries the delegate side-condition $\mathrm{pot}(\mathit{child}) \leq q$; without it the equivalence is false, e.g.* $\mathsf{delegate}(0, \mathsf{call}(1))$*), closing the relational↔functional gap rather than leaving it on paper.*

4.3 Lemma 2 (one-step preservation, strengthened)

Lemma. *If $p \vdash e : \diamond ; p'$ and $\langle e, g\rangle \to \langle e', g'\rangle$ with $d := g' - g$, then (i) $d \leq p$ and (ii) $\exists r \geq p'$ with $p - d \vdash e' : \diamond ; r$.*

The clause $r \geq p'$ lets (T-Seq) recompose.

Proof. By cases (inversion):

- $\mathsf{call}(c) \to \mathsf{skip}$, $d = a \leq c$: $p \geq c$, $p' = p - c$; (i) $a \leq c \leq p$; (ii) $p - a \vdash \mathsf{skip} : \diamond ; p - a$, $r = p - a \geq p - c = p'$.
- $\mathsf{skip}; e_2 \to e_2$, $d = 0$: $p_1 = p$, $p \vdash e_2 : \diamond ; p'$; $r = p'$.
- $e_1; e_2$, $e_1 \to e_1'$, $d \geq 0$: (T-Seq) $p \vdash e_1 : \diamond ; p_1$, $p_1 \vdash e_2 : \diamond ; p'$; IH on e_1: $d \leq p$, $\exists r_1 \geq p_1$. $p - d \vdash e_1' : \diamond ; r_1$; Lemma 1 raises e_2 from p_1 to r_1: $r_1 \vdash e_2 : \diamond ; p' + (r_1 - p_1)$; (T-Seq) gives $r = p' + (r_1 - p_1) \geq p'$.
- $\mathsf{if}\ e_1\ e_2 \to e_i$, $d = 0$: $p \vdash e_i : \diamond ; p_i$, $p_i \geq \min(p_1, p_2) = p'$; $r = p_i$.
- $\mathsf{loop}(n+1, e) \to e; \mathsf{loop}(n, e)$, $d = 0$: body cost $b = b_e$, $p \geq (n+1)b + p'$; by Lemma 0 (normal form of body, raised to p, valid as $p \geq (n+1)b \geq b$) $p \vdash e : \diamond ; p - b$; (T-Loop) (fuel n) $p - b \vdash \mathsf{loop}(n, e) : \diamond ; p - (n+1)b$; (T-Seq) $p \vdash e; \mathsf{loop}(n, e) : \diamond ; p - (n+1)b$; $r = p - (n+1)b \geq p'$. $\mathsf{loop}(0, e) \to \mathsf{skip}$: $r = p \geq p'$.
- $\mathsf{delegate}(q, e) \to e$, $d = 0$: $q \vdash e : \diamond ; q'$, $p \geq q + p'$; Lemma 1 raises child $q \to p$: $p \vdash e : \diamond ; p - q + q'$, $r = p - q + q' \geq p - q \geq p'$. ∎

Note on the `delegate` *case (the q' slack).* The static rule **(T-Del) debits the parent's residual by the full** q — the parent declares the transfer lost irrevocably (the linear split). The operational step, however, **inlines** e **into the parent's single global gas counter** (there is no separate child process to "burn" its unused budget), so when we recompose, the child's unspent potential q' is still accounted. That is why $r = p - q + q' \geq p - q$. The invariant only needs $r \geq p'$, so the q' slack is sound *over*-accounting, never double-spend. The Lean model is strictly more conservative: $\mathrm{pot}(\mathsf{delegate}\ q\ e) = q$ unconditionally, never crediting q' back — so the mechanized bound is q, exactly the static debit, with no reliance on the recomposition slack.

4.4 Theorem (gas bound — a safety invariant on every reachable configuration)

Theorem (Gas bound)**.** *For all g_0: if $p \vdash e : \diamond ; \_$ and $\langle e, g_0\rangle \to^* \langle e'', g\rangle$, then $g - g_0 \leq p$ (with $g_0 = 0$: $g \leq p$).*

Proof. By induction on the number m of steps. $m = 0$: $0 \leq p$. $m \to m+1$: $\langle e, g_0\rangle \to \langle e_1, g_1\rangle \to^* \langle e'', g\rangle$; by Lemma 2, $d_1 = g_1 - g_0 \leq p$ and $p - d_1 \vdash e_1 : \diamond ; \_$; by IH on the m-step subtrace (incoming $p - d_1$), $g - g_1 \leq p - d_1$; summing, $g - g_0 \leq d_1 + (p - d_1) = p$. ■

This is a **safety property**: it holds at every reachable configuration $\langle e'', g\rangle$ and **its proof never appeals to termination** (it is the transitive lift of the per-step credit invariant $g + \Phi$ non-increasing, Lemma 2). The weak fragment is in fact strongly normalizing (§5), so no divergent trace exists *here*, and the Lean theorem quantifies over the (necessarily finite) `Steps` reachable from $\langle e, 0\rangle$. The payoff of proving a *termination-free* invariant — rather than deriving the bound as a corollary of termination — is methodological: the **same $g+\Phi$ argument would carry over** to an extension with genuine non-termination (e.g. fuel-free recursion), since it never uses strong normalization. We flag that this carry-over is an *observation about the technique*, **not a mechanized claim**: the Lean covers exactly this fragment. The theorem `steps_sound` (`Steps` $e\ 0\ e'\ g' \to$ `WellTyped` $e \to g' \leq$ `pot` e) is the safety statement, quantified over all reachable $\langle e', g'\rangle$.

Vector resources, mechanized (arbitrary k). The companion file `lean/TypedResourcesVec.lean` repeats the development with resources as vectors $\mathrm{Res}\,k = \mathrm{Fin}\,k \to \mathbb{N}$ (so $k = 3$ is $\langle$tokens, calls, \$$\rangle$): pot, WellTyped, the instrumented step and the cap axiom ($\forall i.\, a_i \leq c_i$) all lift pointwise, with branch as pointwise max and loop as scalar-times-vector. The same theorem is proved for *every* dimension,

$$\texttt{vsteps\_sound}: \texttt{VSteps}\ e\ \mathbf{0}\ e'\ g' \to \texttt{vWellTyped}\ e \to \forall i.\ g'_i \leq (\mathrm{pot}\, e)_i,$$

i.e. a well-typed vector workflow run from the zero vector spends at most its declared potential in *every* component, on any trace. The worked example of §2 instantiates at $k = 3$ with $\langle$tokens, calls, cents$\rangle$ caps and is checked by computation ($\mathrm{pot}_{\mathrm{tokens}} = 16000$, $\mathrm{pot}_{\mathrm{calls}} = 5$, $\mathrm{pot}_{\mathrm{cents}} = 275$). This closes the "multi-resource is only a pointwise paper extension" gap: the lift is machine-checked, with the same kernel axioms as the scalar theorem (`[propext, Quot.sound]`).

5 Progress, termination, and what the type adds over a runtime tracker

Measure for termination. Use the lexicographic order on pairs (W, S), where W is fuel-weighted work and S is syntactic size. Define $W(\mathsf{skip}) = 0$, $W(\mathsf{call}(c)) = W(\mathsf{tool}(c)) = 1$,

$W(e_1; e_2) = W(e_1) + W(e_2)$, $W(\mathsf{if}\ e_1\ e_2) = \max(W(e_1), W(e_2))$, $W(\mathsf{delegate}(q, e)) = W(e)$, and $W(\mathsf{loop}(n, e)) = n \cdot (W(e) + 1)$ (the $+1$ per iteration is the unrolling overhead). $S(e)$ is the number of AST nodes (always ≥ 1).

Lemma (Decrease — mechanized as `step_decreases` in Lean)**.** *Every operational rule strictly decreases* (W, S) *lexicographically.*

The Lean proof checks all nine rule cases, so the informal reasoning below is backed by the machine, not just prose:

- $\mathsf{call}(c) \to \mathsf{skip}$: W drops $1 \to 0$.
- $\mathsf{loop}(n+1, e) \to e; \mathsf{loop}(n, e)$: W drops by **exactly 1** — LHS $W = (n+1)(W(e)+1)$, RHS $W = W(e) + n(W(e)+1)$, so LHS$-$RHS $= 1$. The unfold **duplicates** e **syntactically, so** S **grows here**; this is harmless because W (the higher-priority component) strictly decreases — standard lexicographic pattern, where S only governs the W-constant steps.
- The W-non-increasing steps — $\mathsf{skip}; e \to e$, $\mathsf{if}\ e_1\ e_2 \to e_i$ (where $W(e_i) \leq \max(W(e_1), W(e_2)) = W(\mathsf{if})$, so W drops or stays equal; if equal, S drops), $\mathsf{loop}(0, e) \to \mathsf{skip}$, $\mathsf{delegate}(q, e) \to e$ — each strictly drop S when W is unchanged (they remove an AST node; $W(\mathsf{delegate}(q, e)) = W(e)$ makes the delegate step W-equal, S-down).
- $e_1; e_2 \to e_1'; e_2$ decreases (W, S) by IH on e_1 (lifted through $W(e_1; e_2) = W(e_1) + W(e_2)$).

Since $<_{\text{lex}}$ on $\mathbb{N} \times \mathbb{N}$ is well-founded, the fragment is strongly normalizing. ■ (`step_decreases` mechanizes the per-rule decrease; the well-foundedness step is standard.)

Progress. Every typable $e \neq \mathsf{skip}$ steps (by inversion: call/tool/if/loop/delegate have a rule; $e_1; e_2$ reduces e_1 or consumes a leading skip). ■

What is and isn't mechanized. Machine-checked in `lean/TypedResources.lean`: (1) the §4 cost-soundness safety invariant `steps_sound`; (2) **Lemma 0** — the relational↔functional equivalence `hastype_iff_pot` : `HasType` $p\ e\ p' \leftrightarrow$ (`WellTyped` $e \wedge$ `pot` $e \leq p \wedge p' = p -$ `pot` e), so the bridge from the §3 typing rules to pot is no longer on paper; (3) **the §5 termination measure** `step_decreases` : `Step` $e\ g\ e'\ g' \to$ `Lex` $e'\ e$, i.e. every operational step strictly decreases the lexicographic (`Wm`, `Sz`) — the load-bearing content of strong normalization. Also machine-checked, in `lean/TypedResourcesVec.lean`: (4) `vsteps_sound` — the same cost-soundness theorem for an arbitrary-dimension resource vector (§4). `steps_sound`, `hastype_iff_pot`, and `vsteps_sound` use only `[propext, Quot.sound]` (constructive); `step_decreases` additionally uses `Classical.choice`; the affine layer's `no_double_spend` (in `lean/Affine.lean`) uses `[propext]`. Still **pen-and-paper**: the final step from `step_decreases` to "no infinite reduction" (standard well-foundedness of $<_{\text{lex}}$ on $\mathbb{N} \times \mathbb{N}$), progress, the fusion of the two layers, and the §3.2 billing facts. So the "*completes* within budget" upgrade now rests on a mechanized measure-decrease plus a standard well-foundedness step; the headline *mechanized* guarantee remains the safety bound "never *spends* more than the declared potential."

A runtime budget with a kill-switch gives safety only — it aborts at meter B, after spending B, possibly mid-task. The type gives more, *ahead of time*:

- **(a)** *Ex-ante rejection of unbounded loops.* A loop without a fuel literal does not type; the canonical runaway is a type error at check time, not a runtime abort after overspending. (Light — syntactic — but a tracker cannot reject it ex-ante.)

- **(b)** *Static compositional conservation for sequential/nested delegation.* (T-DEL) gives $\Sigma\, q_i \leq p$ before execution. *Scope:* sequential/nested only; the concurrent tree needs `par` + interleaving (§8).
- **(c)** *Completes consuming $\leq P$ tokens.* By strong normalization + progress + the gas bound, a well-typed workflow with potential p **terminates and consumes $\leq p$** — "I will finish within P tokens." A tracker gives only "aborts at B": you pay B for a half-finished result. **Two conditions, both explicit:** (i) the cap axiom holds (§3.2) — else per-call cost may exceed c; (ii) the declared window W upper-bounds the re-sent context at every iteration (§3.1) — if W underestimates real context (prompt caching, large tool outputs, retries that regrow context), the token bound breaks though the calculus stays sound. The dollar version additionally needs the cap axiom's billing form. We therefore state the guarantee in tokens/tool-calls, conditional on (i)–(ii).

What the fragment does not give. No-double-spend of a context resource is not a consequence of the pure-potential rules; it needs affine handles with linear split. That orthogonal property is exactly what Khan's affine layer targets — and what our separate `lean/Affine.lean` development mechanizes in its core form (§8, `no_double_spend`).

6 Mapping to frameworks: the checker is a linter, not a new DSL

Construct	LangGraph	OpenAI Agents SDK	CrewAI	Temporal
$\mathsf{call}(c)$	model cap	`max_output_tokens`	model cap	activity
$\mathsf{loop}(n, \cdot)$	`recursion_limit`	`max_turns`	`max_iter`	retry policy
$\mathsf{delegate}(q, \cdot)$	`Send`/subgraph	handoff	hierarchical	child workflow
$\mathsf{par}(\cdot)$ †roadmap	parallel `Send`	parallel tools	—	parallel children

The seven constructs type annotations developers already write; a checker enters as a *linter over existing config*, not a language to adopt. (`par` is roadmap, marked †; it is not in the calculus.)

`gasket`: implementation + empirical study. The calculus is realized as an open-source static checker, `gasket` (Apache-2.0; pure AST, never executes the analysed code). We ran it over a frozen corpus of **254 unique agent-workflow graphs** harvested from **45 public, production-grade repositories** ($\geq$10$\star$ or CI/tests, anti-tutorial filtered, structurally de-duplicated). Findings, under a taxonomy fixed before annotation (Appendix B):

Outcome	Share	Reading
maps onto the calculus (certifiable + default-dependent + rejected)	**76%**	the sequential core covers the bulk of real graph structure
of *cyclic* workflows: bound is only a framework default, or absent	**88%**	most "protected" loops rely on a default — often a **coarse 1000-superstep** LangGraph limit (v1.0.6+), too loose to be a meaningful budget
LLM calls with an explicit token cap	**12%**	dollar bounds need cap inference (a `gasket caps` feature)
outside the calculus (`Send` fan-out, interrupts, hierarchical, dynamic goto, subgraph-as-node)	the rest	the empirically-ranked roadmap for the calculus (§8)

Disclosure (the study's own limits): ∼80% LangGraph (CrewAI/Agents-SDK samples small); ∼5.6 units/repo clustering; public-GitHub visibility bias; LangGraph version assumed ≥1.0.6 where undeclared. This is the kind of evidence — a tool plus a measured corpus — that grounds the framing rather than asserting it; the gap inventory is the calculus's roadmap, not a hidden weakness.

7 Related work

- **Khan [10]** (Jun 2026): 63-incident catalogue + affine (Rust) mitigation; source-level ownership enforced by the borrow checker (**pen-and-paper, not mechanized**); the aggregate cost bound is Proposition 1 (conditional on estimator assumption A1) + empirical validation (382 live sessions), not a proof; **binary-level** soundness left open (Conjecture 1). *We give a machine-checked, operational cost-soundness via potential; we do not touch the binary gap, and our affine layer (§8) mechanizes a thin no-double-spend ledger invariant inspired by the ownership issue Khan targets (full borrow-style ownership is roadmap, not claimed here).*

- **Graded / cost-aware type systems.** Resource-Bounded Type Theory [14] (Dec 2025) proves syntactic cost-soundness for a recursion-free fragment over an abstract resource lattice; its dependent MLTT variant [15]; `calf` [12]; RelCost [3]; Granule [13]. *None is agent/token-specific. Our contribution is the LLM coupling — the cap axiom, the* $\langle tokens, calls, \$\rangle$ *tuple, per-provider billing, framework map — not the potential/grading method.*

- **AARA** (Hofmann–Jost [9]; "Two Decades of AARA" [8]): the potential method; soundness formulations valid for non-terminating executions. *Our fragment is the degenerate linear case.*

- **Resource-aware session types** (Das–Hoffmann–Pfenning [5]; digital contracts [4]): potential over message-passing; potential↔gas. *Our delegation split adapts this to sub-agents.*

- **Agent Contracts [1]** (AAMAS 2026): runtime multi-dimensional bounds with conservation laws; admits a single call can exceed the budget. *Our conservation corollary is its static counterpart.*

- **The LLMbda calculus [6]** (Gordon, Sands, Feb 2026): λ-calculus for agentic programming with information-flow control. *Complementary (info-flow vs resources); LLMbda has no potential/grading, so budget-typing is not a free extension of it — integrating the two is roadmap (§8).*

- **λA / typed agent-composition calculi** [16] (2025–2026): typed calculi for LLM-agent composition with bounded fixpoints, type safety, termination, and lint soundness. *Closest in spirit (a typed calculus for agent composition with bounded loops); they target safety/termination of composition, whereas we target a mechanized* ***aggregate cost*** *bound under an explicit per-call billing axiom. The two are orthogonal and composable.*

8 The affine handle layer (no-double-spend), and roadmap

The potential calculus bounds *how much* a workflow spends. Its orthogonal half — *what* it spends, i.e. that each **context resource** (a session, a lock, a sub-agent handle) is used at most once — is the property at the heart of the ownership discipline Khan enforces with the Rust borrow checker. We mechanize **a minimal core of that property** (affine no-double-spend) as a self-contained layer (`lean/Affine.lean`). *We are precise about what this is and is not:* it is a syntactic

affine discipline over named handles proving the consumption ledger stays duplicate-free; it is **not** the full borrow apparatus — there is no explicit ownership context Γ with membership checks, no move/borrow distinction, no aliasing or handle allocation. Those are the genuine remaining content of "mechanizing Khan's ownership" and are roadmap, not claimed here.

A handle expression consumes named handles; `seqH` **splits** the available handles between its two sides (a handle may appear in at most one), while `branchH` **shares** them (only one branch runs). Affine well-typedness `linear` enforces the split. The instrumented semantics records consumed handles in a ledger, and the central theorem is

> `no_double_spend` : `linear` $e \to$ `disj` (`handles` e) $u \to$ `nodupL` $u \to$ `HSteps` $e\ u\ e'\ u' \to$ `nodupL` u'

— a well-typed affine expression, run from a fresh ledger, **never records the same handle twice on any trace**. `#print axioms no_double_spend` reports `[propext]` only (constructive). The proof is a preservation invariant: each step keeps the remaining expression affine, its handles disjoint from the ledger, and the ledger duplicate-free.

Alongside §4, this gives a machine-checked aggregate cost bound (the half Khan leaves to Proposition 1 + empirics) *and* a thin affine no-double-spend ledger invariant (a much smaller property than the full borrow-style ownership Khan enforces with Rust). The aggregate-cost mechanization specialized to per-call-capped agent workflows we believe is the first; the affine layer is deliberately minimal, not a full borrow system. The two layers compose as a product judgment $p; \Gamma \vdash e \dashv p'; \Gamma'$ (numeric potential $p \times$ affine context Γ), each component independent.

Still roadmap: promoting the affine layer to a **full ownership type system** — explicit context $\Gamma \vdash e \dashv \Gamma'$ with membership checks on `useH`, move/borrow distinction, aliasing and handle allocation (this is the substance of Khan's borrow discipline that the thin layer here does not capture); unifying the two layers in a *single* `Expr` (here they are separate calculi that compose, not one fused syntax) — note the **multi-resource metatheory** over $\mathbb{N}^k$ is no longer roadmap: it is mechanized for arbitrary k (`vsteps_sound`, §4); **`par`/`retry`** with an interleaving semantics; **expected-cost** via probabilistic AARA (Ngo–Carbonneaux–Hoffmann [11]), with its subtleties (optional stopping, supermartingales); **inferred windows** W; and **integration with LLMbda** for a combined information-flow + resource type system.

---

*Artifacts. (1) A **Lean 4 mechanization** (`lean/TypedResources.lean`, self-contained, no Mathlib, Lean v4.30.0) of the calculus: the §4 theorem*

> `steps_sound` : `Steps` $e\ 0\ e'\ g' \to$ `WellTyped` $e \to g' \le$ `pot` e

*(a well-typed workflow from zero gas spends $g' \le$ `pot` e at every reachable configuration — every finite prefix of any trace — where `Step.call`/`Step.tool` encode the cap axiom $a \le c$; proved via the single-step credit invariant `step_sound` and its transitive lift `steps_credit`), plus the Lemma 0 equivalence `hastype_iff_pot` and the §5 measure decrease `step_decreases`. `#print axioms steps_sound` reports only `[propext, Quot.sound]` — Lean's two foundational kernel axioms; no `sorryAx`, no `Classical.choice`, so the proof is constructive. This is the machine-checked claim. (2) The **vector mechanization** (`lean/TypedResourcesVec.lean`): `vsteps_sound`, the same cost-soundness theorem for an arbitrary-dimension resource vector* $\mathrm{Fin}\, k \to \mathbb{N}$ *($k = 3$ is $\langle$tokens, calls, \$$\rangle$), axioms `[propext, Quot.sound]` (§4). (3) The **affine layer** (`lean/Affine.lean`): `no_double_spend`, axioms `[propext]` (§8). (4) A Python sanity*

*harness (`check.py`) on a fixed 5-program battery, which (i) confirms the no-fuel runaway loop does not type, (ii) confirms sequential delegation conservation, and (iii) reports zero $g \leq p$ violations over random cost assignments. Note (ii)–(iii) only test that the executable rules agree with the paper: the generator enforces $a \leq c$ by construction, so the harness **cannot** witness a cap violation — it validates rule/code consistency, not soundness under an adversarial provider. Soundness is the Lean theorem, not an empirical claim.*

References

[1] Agent contracts: A formal framework for resource-bounded autonomous AI systems, 2026. arXiv:2601.08815. AAMAS 2026.

[2] Quentin Carbonneaux, Jan Hoffmann, and Zhong Shao. Compositional certified resource bounds. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2015)*, pages 467–478. ACM, 2015.

[3] Ezgi Çiçek, Gilles Barthe, Marco Gaboardi, Deepak Garg, and Jan Hoffmann. Relational cost analysis. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL 2017)*, pages 316–329. ACM, 2017.

[4] Ankush Das, Stephanie Balzer, Jan Hoffmann, Frank Pfenning, and Ishani Santurkar. Resource-aware session types for digital contracts, 2019. arXiv:1902.06056. Also in IEEE CSF 2021.

[5] Ankush Das, Jan Hoffmann, and Frank Pfenning. Work analysis with resource-aware session types. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2018)*, pages 305–314. ACM, 2018.

[6] Andrew D. Gordon and David Sands. The LLMbda calculus: A lambda-calculus for agentic programming with information-flow control, 2026. arXiv:2602.20064.

[7] Tingxu Han, Zhenting Wang, Chunrong Fang, Shiyu Zhao, Shiqing Ma, and Zhenyu Chen. Token-budget-aware LLM reasoning, 2024. arXiv:2412.18547.

[8] Jan Hoffmann and Steffen Jost. Two decades of automatic amortized resource analysis. *Mathematical Structures in Computer Science*, 32(6):729–759, 2022.

[9] Martin Hofmann and Steffen Jost. Static prediction of heap space usage for first-order functional programs. In *Proceedings of the 30th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 2003)*, pages 185–197. ACM, 2003.

[10] Sajjad Khan. Token budgets: An empirical catalog of 63 LLM-agent budget-overrun incidents, with an affine-typed Rust mitigation as a case study, 2026. arXiv:2606.04056.

[11] Van Chan Ngo, Quentin Carbonneaux, and Jan Hoffmann. Bounded expectations: Resource analysis for probabilistic programs. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2018)*, pages 496–512. ACM, 2018.

[12] Yue Niu, Jonathan Sterling, Harrison Grodin, and Robert Harper. A cost-aware logical framework, 2021. arXiv:2107.04663. Also in Proc. ACM Program. Lang. 6(POPL), 2022.

[13] Dominic Orchard, Vilem-Benjamin Liepelt, and Harley Eades III. Quantitative program reasoning with graded modal types. *Proceedings of the ACM on Programming Languages*, 3(ICFP):110:1–110:30, 2019.

[14] Resource-bounded type theory: Compositional cost analysis via graded modalities, 2025. arXiv:2512.06952.

[15] Resource-bounded Martin-löf type theory: Compositional cost analysis for dependent types, 2026. arXiv:2601.10772.

[16] (see citation in paper). A typed calculus for LLM-agent composition (lambda-a), 2026. cited as closest typed agent-composition work.

A Reproducibility of the `gasket` empirical study (§6)

Artifact. The checker (`gasket`, Apache-2.0), the frozen corpus manifest (the 45 source repositories with the exact commit SHA pinned per repository), the harvest/normalization script, the annotation taxonomy, and the per-unit results are archived with the paper (Zenodo record; `gasket` source at the URL in the README). `gasket` is pure-AST and never executes the analysed code, so the study is byte-deterministic: re-running the checker over the pinned corpus reproduces the per-unit verdicts exactly.

Unit of analysis. A *graph unit* is one compiled agent graph (one `StateGraph`/`Crew`/`Agent` construction site), *not* a file or a repository; a file may contribute several. Units are structurally de-duplicated by a canonicalized AST fingerprint (node/edge shape with literals erased) so that copy-pasted graphs are not double-counted. The corpus is 254 unique units from 45 repositories (mean ≈ 5.6 units/repo).

Repository selection. Public GitHub repositories using LangGraph, the OpenAI Agents SDK, or CrewAI, filtered to $\geq 10\star$ *or* the presence of CI/tests (a production-grade proxy), with tutorial/example/quickstart repositories excluded by path and metadata heuristics. We disclose the resulting skew: $\approx 80\%$ of units are LangGraph (the CrewAI and Agents-SDK samples are small), and visibility is biased toward public GitHub.

Operational definitions (fixed before annotation).

- *Maps onto the calculus*: every loop/branch/delegation/call site in the unit is expressible by one of the seven constructs (a unit with a `Send` fan-out, an interrupt, a hierarchical sub-crew, a dynamic `goto`, or a subgraph-as-node is counted as *outside*).
- *Cyclic workflow*: the graph contains a back-edge / fixpoint (`recursion_limit`, `max_turns`, `max_iter`, or an explicit cycle).
- *Framework default*: the only bound on a cycle is the framework's default cap, not a developer-set literal. Verified against primary docs: LangGraph `recursion_limit` default = 1000 supersteps (v1.0.6+; *not* the older 25), CrewAI `max_iter` = 20, OpenAI Agents SDK `max_turns` = 10 (`None` disables it). Because a superstep may execute several nodes, the implied bound is $n \cdot \Sigma_i b_i$ (sum over nodes), not $n \cdot \max$.
- *Explicit token cap*: the model-call site sets a per-call output-token ceiling (`max_output_tokens`, `max_completion_tokens`, or `max_tokens`).

Manual validation. Every unit the checker classified as *rejected/outside* was re-examined by hand (not a fixed sample), which surfaced and fixed five harvester bugs before the final run — a variable (non-`None`) `max_turns` misread as unbounded; a REPL `input()` loop misread as a runaway; a one-argument `add_node(fn)` whose node name is the function name; `NodeInterrupt`; and a subgraph used as a node. The headline shares (**76%** maps, **88%** of cyclic units default-bound-or-absent, **12%** with an explicit token cap) are reported after these fixes. Residual disclosure: false-*rejection* 0/3 on the spot-checked certifiable set (a wide interval at $N = 3$); one residual false-*accept* out of ten on the audited accept set.

B Taxonomy and `gasket` examples (§6)

Taxonomy (fixed before annotation). Each unit is placed in exactly one leaf of a strict tree: CERTIFIABLE (every call has an explicit cap and every cycle a literal fuel) / DEFAULT-DEPENDENT (maps, but a cycle's only bound is a framework default) / REJECTED (maps structurally but a required bound is missing, e.g. a fuel-free loop) / OUTSIDE (a construct not in the calculus). The empirically-ranked OUTSIDE gaps, most to least frequent, are: interrupt / human-in-the-loop (19), unresolved bound (16), all-nodes-dynamic (10), `Send` fan-out (9), hierarchical (5), dynamic `goto` (4), subgraph-as-node (1) — this ranking *is* the calculus's roadmap (§8).

Three representative cases (`gasket check` verdict in brackets):

1. *Certifiable* [PASS]. A linear pipeline with capped calls and an explicit `recursion_limit` literal: `call(2000); recursion_limit=8` $\Rightarrow$ pot finite, residual $\geq 0 \Rightarrow$ a signed certificate.

2. *Default-dependent* [WARN]. A LangGraph cycle with *no* developer-set `recursion_limit`: it "types" only against the coarse default of 1000 supersteps — `gasket` reports the bound as *default-derived,* $1000 \cdot \Sigma_i b_i$, i.e. a bound that exists on paper but is far too loose to be a budget.

3. *Rejected* [FAIL]. A "retry until done" loop with no fuel literal (`while not done: agent.invoke(...)`): no `loop` rule applies, so the unit does not type — the canonical runaway is a static error, surfaced at check time rather than after an overspend.